

Design Document: Operationalizing Billion-Scale URL Collection

1. Objective

The goal of this system is to operationalize the collection of metadata and page content for billions of URLs for a given year and month. The input can arrive either as large text files or from a MySQL table. The output of the system should be a reliable, queryable, and auditable dataset containing crawl metadata, raw content locations, crawl status, errors, and operational metrics.

The crawler code developed in Phase 1 will be productionized by wrapping it inside a distributed, queue-based architecture. Instead of running the crawler as a single process, the system will divide billions of URLs into smaller crawl jobs, distribute them across many workers, store results in scalable storage systems, and continuously monitor progress, failures, cost, and performance.

The high-level architecture contains these major stages:

1. Input sources
 2. Ingestion layer
 3. Queue and scheduler
 4. Distributed crawling
 5. Processing layer
 6. Storage layer
 7. Observability layer
-

2. Input Sources

The system supports two primary input formats.

2.1 Text File Input

For very large URL batches, text files can be uploaded to object storage such as Amazon S3, Google Cloud Storage, or Azure Blob Storage.

Each file contains URLs for a specific year and month, for example:

```
2026/07/urls_part_0001.txt
2026/07/urls_part_0002.txt
2026/07/urls_part_0003.txt
```

Each line contains one URL:

```
https://example.com/page1
https://example.com/page2
https://anotherdomain.com/article
```

This approach is useful because object storage can handle very large files cheaply and reliably.

2.2 MySQL Input

The system also supports URLs stored in MySQL. The table may contain columns such as:

```
id
url
year
month
source
created_at
```

For billion-scale extraction, the system should avoid loading all records at once. Instead, it should read records in chunks using keyset pagination.

Example:

```
SELECT id, url, year, month
FROM source_urls
WHERE year = 2026
      AND month = 7
      AND id > :last_seen_id
ORDER BY id
LIMIT 100000;
```

This is better than offset-based pagination because offset queries become slow as the table grows.

3. Batch Manifest

The first component in the ingestion layer is the Batch Manifest Creator.

A batch manifest is a control record that represents one monthly crawl job. It allows the system to track progress from input ingestion to crawl completion.

Example manifest fields:

```
batch_id: 2026-07-main
year: 2026
month: 07
source_type: mysql / text_file
source_location: s3://bucket/2026/07/ or mysql.source_urls
expected_url_count: 5,000,000,000
status: CREATED / INGESTING / CRAWLING / COMPLETED / FAILED
created_at
started_at
completed_at
```

The batch manifest is important because billion-scale systems need strong progress tracking. Without a manifest, it becomes difficult to know how many URLs were received, how many were valid, how many were crawled, and how many failed.

4. Ingestion Layer

The ingestion layer prepares raw input URLs before they are sent to the crawl queue.

4.1 URL Validator

The URL Validator checks whether each URL is valid and crawlable.

Validation includes:

- URL is not empty.
- URL has a valid scheme, usually `http` or `https`.
- URL has a valid hostname.
- URL is not malformed.
- URL length is within acceptable limits.
- Unsupported schemes such as `ftp`, `mailto`, and `javascript` are rejected.

Invalid URLs are not sent to the crawler. They are stored with a failure reason so the final output remains auditable.

Example status:

```
SKIPPED_INVALID_URL
```

4.2 URL Normalizer

The URL Normalizer converts URLs into a consistent format. This reduces duplicate crawling and improves storage consistency.

Normalization rules may include:

- Lowercase the hostname.
- Remove URL fragments such as `#section`.
- Remove default ports such as `:80` for HTTP and `:443` for HTTPS.
- Normalize trailing slashes carefully.
- Sort query parameters where safe.
- Convert internationalized domain names into a standard format.

Example:

```
Original URL:  
HTTPS://Example.com:443/page?id=10#reviews
```

```
Normalized URL:  
https://example.com/page?id=10
```

4.3 Deduplication Service

At billion scale, deduplication is very important because every duplicate URL wastes compute, network, queue, and storage cost.

The deduplication service generates a hash from the normalized URL.

Example:

```
normalized_url = https://example.com/page?id=10
normalized_url_hash = sha256(normalized_url)
```

The design uses two levels of deduplication:

```
Fast check:
Redis / Bloom Filter
```

```
Final correctness:
Durable dedupe table
```

Redis or RedisBloom can be used as a fast membership check. This reduces expensive database lookups. However, Redis should not be the only source of truth because Bloom filters can have false positives and Redis may not be ideal as the final audit store for billions of records.

The durable dedupe table should enforce uniqueness using:

```
batch_id + normalized_url_hash
```

If the URL already exists for the batch, it is marked as duplicate and skipped. If it is new, it is inserted into the dedupe table and pushed to the main URL queue.

5. Queue and Scheduler Layer

After ingestion, valid and deduplicated URLs are sent to the queue layer.

The queue layer decouples ingestion from crawling. This is important because ingestion may produce URLs faster than the crawler can process them.

Recommended queue options:

- Kafka
- AWS SQS
- Google Pub/Sub
- Azure Service Bus

5.1 Main URL Queue

The main queue contains crawl jobs ready to be processed.

Example message:

```
{
  "batch_id": "2026-07-main",
```

```
{
  "url_id": "123456",
  "original_url": "https://example.com/page",
  "normalized_url": "https://example.com/page",
  "normalized_url_hash": "abc123",
  "year": 2026,
  "month": 7,
  "attempt": 1
}
```

5.2 Domain Scheduler

The Domain Scheduler controls how URLs are distributed to crawler workers.

This component is important because if the system blindly crawls billions of URLs, it may overload individual domains. The scheduler should group or partition URLs by domain and enforce domain-level limits.

For example:

```
example.com: max 5 requests per second
news-site.com: max 10 requests per second
small-blog.com: max 1 request per second
```

This protects target websites and also reduces crawler failures caused by throttling, blocking, or timeouts.

5.3 Retry Queue

The retry queue stores URLs that failed due to temporary issues.

Retryable failures include:

- Timeout
- DNS temporary failure
- Connection reset
- HTTP 429
- HTTP 500
- HTTP 502
- HTTP 503
- HTTP 504

The retry policy should use exponential backoff with jitter.

Example:

```
Attempt 1: immediate
Attempt 2: retry after 5 minutes
Attempt 3: retry after 30 minutes
Attempt 4: retry after 2 hours
After max attempts: send to dead letter queue
```

5.4 Dead Letter Queue

The dead letter queue stores URLs that cannot be successfully processed after all retry attempts or have permanent failures.

Permanent failures include:

- Invalid URL
- Robots.txt disallowed
- HTTP 404
- HTTP 410
- Too many redirects
- Unsupported content type
- Maximum retry attempts exceeded

These records should not disappear. They should be stored in an error table with the reason for failure.

6. Distributed Crawling Layer

The distributed crawling layer runs the Phase 1 crawler code inside many containerized workers.

The worker pool can run on:

- Kubernetes
- AWS ECS
- Google Kubernetes Engine
- Azure Kubernetes Service

Each crawler worker should be stateless. This allows the system to scale workers up or down based on queue depth.

6.1 Crawler Worker Pool

Each worker performs the following steps:

1. Pull a URL message from the queue.
2. Check robots.txt rules.
3. Apply domain rate limiting.
4. Fetch the URL.
5. Capture HTTP response data.
6. Send the response to the processing layer.

7. Update crawl status.

The worker should capture:

```
HTTP status code
Final URL after redirects
Redirect count
Response headers
Content type
Content length
Crawl latency
Error reason, if any
```

6.2 Robots.txt Cache

The Robots.txt Cache can be implemented using Redis.

Before crawling a domain, the worker checks whether robots.txt rules are already cached.

Flow:

```
Worker checks Redis for domain robots.txt
If present:
  apply cached rules
If missing:
  fetch robots.txt
  parse rules
  store parsed result in Redis with TTL
```

Recommended TTL:

```
Successful robots.txt fetch: 24 hours
Failed robots.txt fetch: 1 to 6 hours
```

This avoids repeatedly fetching the same robots.txt file from the same domain.

6.3 Domain Rate Limiter

The Domain Rate Limiter ensures the crawler does not send too many requests to the same domain.

This can also be implemented using Redis because rate limit counters need to be shared across many workers.

Example:

```
Key: rate_limit:example.com
Limit: 5 requests per second
```

This protects both the crawler system and external websites.

6.4 HTTP Fetcher

The HTTP Fetcher is the component that actually requests the web page.

Recommended fetch settings:

```
Connection timeout: 5-10 seconds
Read timeout: 10-30 seconds
Max redirects: 5
```

Max response size: 5-10 MB
Allowed schemes: HTTP and HTTPS

The fetcher should avoid downloading extremely large files unless those are required. For example, if the project only needs HTML pages, large PDFs, videos, and binary files can be skipped or stored separately.

7. Processing Layer

The processing layer takes the HTTP response and converts it into structured output.

7.1 Metadata Extractor

The Metadata Extractor extracts useful information from the response.

Examples:

```
Page title
Meta description
Canonical URL
Content type
HTTP headers
Status code
Final URL
Language
Page size
Crawl timestamp
```

For HTML pages, the parser can extract title, canonical link, meta tags, and other SEO-relevant fields.

7.2 Content Hasher

The Content Hasher computes a hash of the response body.

Example:

```
content_hash = sha256(response_body)
```

This helps detect duplicate pages. Many different URLs may return the same content, especially if tracking parameters are used.

Content hashing helps optimize storage because duplicate content does not always need to be stored again.

7.3 Schema Validator

The Schema Validator ensures that every crawl result follows the unified schema before being written to storage.

This prevents corrupt or inconsistent records from entering the final dataset.

Validation examples:


```
batch_id must not be null
normalized_url_hash must not be null
crawl_status must be a known enum
completed_at must be present for terminal records
raw_content_path must be present for successful content writes
```

7.4 Result Classifier

The Result Classifier decides the final outcome of each crawl attempt.

Possible statuses:

```
SUCCESS
FAILED_RETRYABLE
FAILED_PERMANENT
SKIPPED_DUPLICATE
SKIPPED_ROBOTS
SKIPPED_INVALID_URL
DEAD_LETTER
```

If the failure is retryable, the message goes to the retry queue.

If the failure is permanent, the message goes to the dead letter queue and error store.

If successful, metadata and content are written to storage.

8. Storage Design

The system separates raw content, metadata, operational status, and error records. This separation keeps the architecture scalable and cost-efficient.

8.1 Raw Content Storage

Raw page content should be stored in object storage such as S3, GCS, or Azure Blob.

Example path:

```
s3://url-crawl-content/year=2026/month=07/domain_hash=ab/url_hash=xyz.html.gz
```

Content should be compressed before storage.

Recommended formats:

```
HTML: .html.gz
JSON: .json.gz
Headers: .json.gz
```

Object storage is suitable because it is cheap, durable, and can handle massive scale.

8.2 Metadata Warehouse

Structured metadata should be stored in an analytical system such as:

```
BigQuery
Snowflake
Redshift
Databricks Delta Lake
```

Apache Iceberg on S3

This allows large-scale querying by month, domain, crawl status, HTTP status, and content type.

The table should be partitioned by:

```
year  
month
```

It can also be clustered or sorted by:

```
domain  
normalized_url_hash  
crawl_status
```

8.3 Operational Status Database

The operational status database tracks batch-level and URL-level progress.

This can be stored in:

```
PostgreSQL  
MySQL  
DynamoDB  
Cassandra  
MongoDB
```

For billions of URLs, a distributed key-value store such as DynamoDB, Cassandra, or ScyllaDB may be better than a single relational database.

This database answers questions like:

```
How many URLs are pending?  
How many completed successfully?  
How many failed?  
Which batch is currently running?  
Which URLs are stuck?
```

8.4 Error Store

The error store contains failed URLs and failure reasons.

Example fields:

```
batch_id  
normalized_url  
normalized_url_hash  
error_type  
error_message  
http_status_code  
attempt_count  
last_attempt_at  
final_status
```

This is useful for debugging, reporting, and possible reprocessing.

9. Unified Data Schema

A unified schema is needed so that output records are consistent regardless of whether the input came from text files or MySQL.

9.1 Main Metadata Table

Example table: `crawl_metadata`

```
CREATE TABLE crawl_metadata (  
  crawl_id STRING,  
  batch_id STRING,  
  source_type STRING,  
  source_id STRING,  
  
  year INT,  
  month INT,  
  
  original_url STRING,  
  normalized_url STRING,  
  normalized_url_hash STRING,  
  domain STRING,  
  registrable_domain STRING,  
  
  crawl_status STRING,  
  http_status_code INT,  
  error_type STRING,  
  error_message STRING,  
  
  requested_at TIMESTAMP,  
  started_at TIMESTAMP,  
  completed_at TIMESTAMP,  
  crawl_latency_ms INT,  
  attempt_count INT,  
  
  final_url STRING,  
  redirect_count INT,  
  content_type STRING,  
  content_length_bytes BIGINT,  
  content_hash STRING,  
  
  title STRING,  
  meta_description STRING,  
  canonical_url STRING,  
  robots_allowed BOOLEAN,  
  
  raw_content_path STRING,  
  headers_path STRING,  
  
  worker_id STRING,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);
```

9.2 Batch Table

Example table: `crawl_batches`

```
CREATE TABLE crawl_batches (  
  batch_id STRING,
```

```
year INT,  
month INT,  
source_type STRING,  
source_location STRING,  
  
total_input_urls BIGINT,  
valid_urls BIGINT,  
invalid_urls BIGINT,  
duplicate_urls BIGINT,  
queued_urls BIGINT,  
completed_urls BIGINT,  
failed_urls BIGINT,  
  
status STRING,  
created_at TIMESTAMP,  
started_at TIMESTAMP,  
completed_at TIMESTAMP  
);
```

9.3 Error Table

Example table: crawl_errors

```
CREATE TABLE crawl_errors (  
  batch_id STRING,  
  normalized_url_hash STRING,  
  normalized_url STRING,  
  
  error_type STRING,  
  error_message STRING,  
  http_status_code INT,  
  attempt_count INT,  
  
  first_failed_at TIMESTAMP,  
  last_failed_at TIMESTAMP,  
  final_status STRING  
);
```

10. SLOs and SLAs

10.1 SLOs

SLOs are internal engineering targets used to measure system health.

Recommended SLOs:

```
Ingestion availability: 99.9%  
Queue durability: 99.99%  
Crawler worker availability: 99.5%  
Metadata write success rate: 99.9%  
Raw content write success rate: 99.5%
```

Processing SLOs:

```
95% of URLs should reach a terminal state within 48 hours.  
99% of URLs should reach a terminal state within 7 days.  
Retry queue should stay below 5% of active workload.
```

Duplicate crawl rate should stay below 1%.
Monitoring data should be delayed by no more than 15 minutes.

A terminal state means:

SUCCESS
FAILED_PERMANENT
SKIPPED_DUPLICATE
SKIPPED_ROBOTS
DEAD_LETTER

10.2 SLAs

SLAs are commitments made to users or downstream consumers.

Recommended SLAs:

Monthly URL batches will be processed within 7 calendar days after input availability.
Final metadata will be available within 24 hours after crawl completion.
Raw content will be available for successfully crawled URLs.
Failed URLs will include a clear terminal failure reason.
Progress dashboards will be available during active crawls.

11. Monitoring Metrics

The observability layer is critical because at billion scale, small failure percentages can still mean millions of failed URLs.

11.1 Ingestion Metrics

Track:

Input URLs received
URLs processed per minute
Invalid URL count
Duplicate URL count
Valid URL count
Ingestion lag
Batch manifest status
MySQL extraction lag
File read failures

11.2 Queue Metrics

Track:

Main queue depth
Retry queue depth
Dead letter queue count
Oldest message age
Consumer lag
Messages processed per second
Queue publish failures
Queue consume failures

11.3 Crawler Metrics

Track:

```
URLs crawled per second
Success rate
Failure rate
Timeout rate
DNS failure rate
TLS failure rate
HTTP status distribution
Redirect count distribution
Average crawl latency
p95 crawl latency
p99 crawl latency
Average response size
Robots.txt blocked count
```

11.4 Storage Metrics

Track:

```
Raw content write success rate
Raw content write latency
Metadata write success rate
Metadata write latency
Storage size growth
Compression ratio
Content deduplication savings
Failed writes
```

11.5 Data Quality Metrics

Track:

```
Missing title percentage
Empty content percentage
Duplicate content hash percentage
Unsupported content type count
Schema validation failures
Records missing raw_content_path
Records missing final_url
```

11.6 Business Progress Metrics

Track:

```
Total URLs in batch
URLs completed
URLs remaining
Successful crawls
Permanent failures
Retryable failures
Estimated time to completion
Cost per million URLs
Projected total cost
```

12. Monitoring Tools

Recommended tools:

```
Prometheus for metrics collection
Grafana for dashboards
OpenTelemetry for traces
ELK / OpenSearch / Loki for logs
Alertmanager for alerting
CloudWatch if using AWS
Cloud Logging and Monitoring if using GCP
```

The main dashboard should show:

```
Batch progress percentage
Queue depth
Crawl throughput
Success/failure rate
Retry queue size
Dead letter queue size
Storage write errors
Estimated completion time
Cost per million URLs
```

Alerts should be created for:

```
Queue depth growing too fast
Retry queue above threshold
Dead letter queue spike
Metadata writes failing
Raw content writes failing
Crawler success rate dropping
p95 latency increasing
No progress for active batch
Worker crash loop
```

13. Cost Optimization

At billion scale, cost optimization must be designed into the system from the beginning.

Key optimizations:

```
Deduplicate before crawling.
Use Redis/Bloom filters to reduce database lookups.
Compress raw content before storage.
Store large raw content in object storage, not databases.
Use lifecycle policies to move old content to cheaper storage.
Use spot or preemptible instances for crawler workers.
Autoscale workers based on queue depth.
Limit maximum response size.
Skip unsupported content types.
Avoid retry storms with exponential backoff.
Batch metadata writes instead of writing one record at a time.
Partition metadata by year and month.
Use content hashes to avoid storing duplicate content.
```

A useful metric is:

```
Cost per million URLs =
```

```
total compute cost + queue cost + storage cost + network cost
divided by processed URL count
```

This metric should be tracked continuously.

14. Reliability Improvements

Reliability is handled through queue durability, idempotent processing, retry logic, and reconciliation.

Important reliability features:

```
Use durable queues.
Use retry queues for temporary failures.
Use dead letter queues for terminal failures.
Make workers stateless.
Use idempotent writes using batch_id + normalized_url_hash.
Use checkpointing during ingestion.
Use domain-level rate limiting.
Use robots.txt caching.
Use worker health checks.
Use autoscaling.
Run reconciliation jobs to find stuck URLs.
```

A reconciliation job should periodically check for URLs that are stuck in `IN_PROGRESS` for too long and requeue them if needed.

Example:

```
If URL status = IN_PROGRESS
and updated_at < current_time - 2 hours
then mark as STALE and requeue
```

15. Performance and Scale

The system scales horizontally.

Main scaling points:

```
More ingestion workers can be added for larger input files.
More queue partitions can be added for higher throughput.
More crawler workers can be added based on queue depth.
Storage scales independently from compute.
Metadata tables are partitioned for faster queries.
```

Capacity planning can be estimated using:

```
required_workers =
target_urls_per_second / urls_per_second_per_worker
```

Example:

```
If each worker crawls 5 URLs per second
and the target is 50,000 URLs per second,
then required workers = 10,000.
```


In practice, throughput depends on network latency, target websites, robots.txt rules, response sizes, and retry rates.

16. Next Steps

To move from Phase 1 code to this production design, I would take the following steps:

1. Containerize the Phase 1 crawler code.
 2. Add structured logging and metrics to the crawler.
 3. Add URL validation and normalization.
 4. Implement URL hashing and deduplication.
 5. Add queue-based crawl execution.
 6. Add retry queue and dead letter queue support.
 7. Add Redis-based robots.txt cache.
 8. Add Redis-based domain rate limiting.
 9. Store raw content in object storage.
 10. Store structured metadata in a warehouse table.
 11. Store batch and URL status in an operational database.
 12. Build Grafana dashboards for progress and failures.
 13. Run a small test with 1 million URLs.
 14. Tune timeout, retry, and rate limit settings.
 15. Scale to 100 million URLs.
 16. Finally scale to billion-level monthly batches.
-

17. Summary

This design turns the Phase 1 crawler into a production-grade distributed URL collection platform. The system accepts URLs from files or MySQL, validates and normalizes them, removes duplicates, distributes crawl work through a durable queue, crawls pages using horizontally scalable workers, processes metadata and content, and stores the output in scalable storage systems.

The most important operational parts of the design are:

```
Batch tracking
Deduplication
Queue-based processing
Domain-aware scheduling
Robots.txt caching
Retry and dead letter handling
Separate raw content and metadata storage
```

SLO/SLA definitions
Monitoring and alerting
Cost tracking

This architecture is designed to be scalable, reliable, cost-aware, and observable enough to handle billions of URLs for monthly crawl workloads.